How LLM APIs Are Served at Scale

Scaler School of Technology | ML System Design and LLMOps | Week 6 — Session 2 | Deep-Dive Case Study: OpenAI, Cohere, and Anthropic

This session: You designed an LLM serving system in W6S1. Today we compare your design to what OpenAI and Cohere actually built. The core question: how do you serve a 175-billion-parameter model to millions of users, in real-time, at low cost, with multi-tenant isolation?

The Unprecedented Scaling Event

ChatGPT launch timeline:

- November 30, 2022: public launch
- December 4, 2022 (5 days later): 1 million users
- February 2023 (2 months later): 100 million users
- Fastest consumer product to 100M users in history

Why this is a hard problem: A single A100 GPU (80GB VRAM) can serve approximately 20–30 concurrent GPT-4 requests, depending on sequence length and quantisation. At 10 million requests per day (which ChatGPT reached quickly), you need hundreds of A100s running continuously *just for GPT-4*. Unlike a web server, you cannot simply spin up more instances of a stateless service — each inference server must hold the full model in GPU memory.

The Two Systems We Are Studying

OpenAI API

Users: Consumer + developer API
Scale: 100M+ monthly users
Model tiering: gpt-3.5 / gpt-4 / gpt-4o
Key design: Stateless, horizontal scaling
Innovation: Prompt caching, token-bucket rate limiting

Cohere Enterprise

Users: Enterprise B2B customers
Scale: Thousands of enterprise tenants
Model tiering: Shared + private deployment
Key design: LoRA for per-customer fine-tuning
Innovation: Private VPC deployment, embedding at scale

The critical difference from recommendation systems: In W5S2 we studied batch pipelines (Spotify runs weekly). LLM serving is *synchronous, online, and autoregressive*. Each token is generated one at a time, and the user is waiting. A recommendation system with a 200ms P99 is fine. An LLM that generates 50 tokens at 200ms per token takes 10 seconds — unacceptable for a conversational interface. Latency is a first-order constraint, not a guardrail.

OpenAI API: Serving at Consumer Scale

Stateless serving · Model routing · Token-bucket rate limiting · Prompt caching

Design 1 | Stateless Serving: Why Context Lives in the Request

Key design decision: OpenAI's API is **fully stateless**. Every API call includes the full conversation context in the request body. The server has no memory between calls. This is the foundational design choice that enables horizontal scaling.

What stateless means:

- Every request contains: system prompt + full conversation history + new user message
- The API server processes the request and returns a response — with no write to any database
- The application developer (your code) is responsible for storing and re-sending conversation history
- A load balancer can route any request to any server — no session affinity required

Why this enables scale: With stateless serving, adding more servers immediately adds more capacity. The only shared state is the model weights (read-only, loaded into GPU memory on startup). No database hotspots, no session locks, no distributed state synchronisation.

The trade-off: Context grows with conversation length. A 20-turn conversation requires sending 20 turns of history on turn 21. This increases input token count — and OpenAI charges for input tokens. Long conversations become expensive. This is a fundamental tension between statelessness (scaling simplicity) and cost efficiency.

Design 2 | Model Routing: One API, Many Model Versions

OpenAI maintains multiple model versions simultaneously: gpt-3.5-turbo (fast, cheap, smaller), gpt-4 (slow, expensive, larger), gpt-4-turbo (faster version of GPT-4), gpt-4o (multimodal). Each runs on different hardware with different capacity.

Explicit routing: The API caller specifies the model in the request (`model: "gpt-4"`). This is direct routing — the API gateway routes to the appropriate cluster.

Implicit routing: Some products (like ChatGPT web) may route requests to different model versions based on load, cost, or A/B test assignment. Users do not see this.

The capacity planning problem:

Model	Relative cost	GPU memory	Concurrent requests per A100
gpt-3.5-turbo	1×	Low	∼100+ (small model, fp16)
gpt-4	30×	High	∼20–30 (large model, quantised)
gpt-4-turbo	10×	Medium-high	∼40–60 (optimised)

Design 3 | Rate Limiting: Token Bucket Algorithm

The problem: Without rate limiting, a single customer running a batch job at midnight could exhaust GPU capacity, degrading service for all other users. Rate limiting protects capacity fairness.

OpenAI's three-dimensional rate limiting:

- **RPM (Requests Per Minute):** Maximum number of API calls per minute
- **TPM (Tokens Per Minute):** Maximum total tokens (input + output) per minute
- **TPD (Tokens Per Day):** Maximum total tokens per day

Token bucket algorithm: A bucket starts full at capacity C (e.g., 90,000 tokens). Each request drains the bucket by its token count. The bucket refills at rate r (e.g., 90,000 tokens per minute = 1,500 tokens per second). If a request arrives when the bucket is empty, it is rejected (HTTP 429 Too Many Requests).

Why token-based, not request-based? Requests vary enormously in size. A 10-token request and a 10,000-token request use very different GPU resources. Limiting by tokens ensures fair resource consumption, not just fair request counts.

Tiered limits: Free tier (limited), Pay-as-you-go (moderate), Enterprise (custom). Enterprise customers negotiate custom limits based on committed spend.

Design 4 | Prompt Caching: Skipping Repeated KV Computation

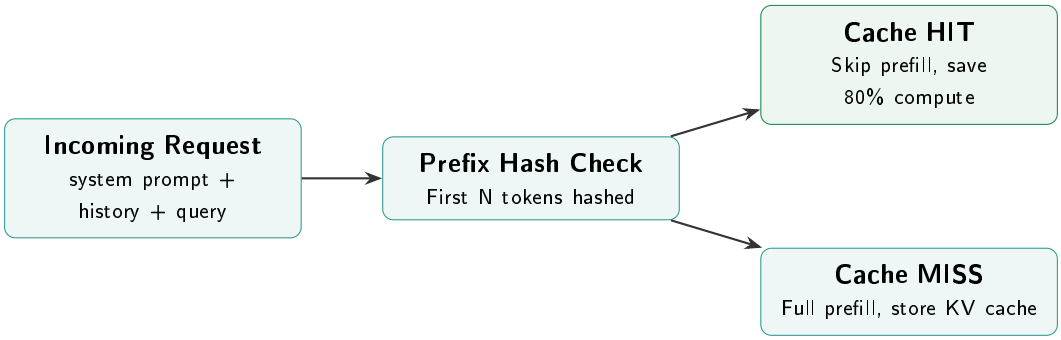
The insight: Many API users (especially in production applications) send the same long system prompt with every request. Without caching, the model re-computes the KV (key-value) attention cache for that system prompt on every single request. Prompt caching skips this computation if the prefix matches a cached version.

How it works mechanically:

1. The API server computes a hash of the first N tokens of the request
2. If this hash matches a cached KV tensor in GPU memory, skip the prefill computation for those tokens
3. Process only the new (uncached) tokens
4. Return the response; the cached KV state persists for a short TTL

Impact: Up to 80% cost reduction for applications with long static system prompts. A customer running a RAG chatbot with a 2,000-token system prompt pays 80% less per query.

The constraint: Caching only works for exact prefix matches. If the system prompt changes by one character, the cache misses. Application developers must structure prompts with the static prefix first and the dynamic content (user query) at the end.



Cohere: Enterprise Multi-Tenant LLM Serving

LoRA for fine-tuning · Private deployment · Embedding at scale · Multi-region

Design 1 | Multi-Tenant Fine-Tuning with LoRA

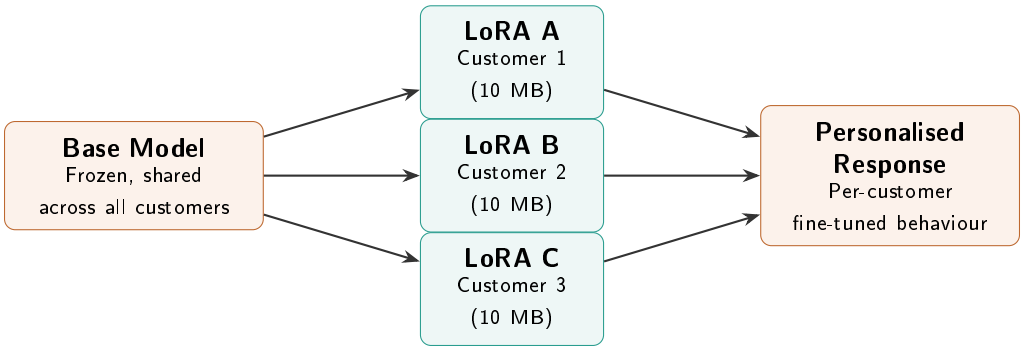
The enterprise problem: Cohere serves thousands of enterprise customers. Each enterprise wants a fine-tuned model on their proprietary data. You cannot load 10,000 different fine-tuned models into GPU memory simultaneously. The solution: LoRA (Low-Rank Adaptation).

What LoRA is: Instead of fine-tuning all parameters of a large model (billions of weights), LoRA injects small trainable weight matrices ("adaptors") into the attention layers. The full model weights stay frozen. Only the LoRA matrices are trained — typically a few million parameters vs. billions in the base model.

The serving architecture:

1. One copy of the base model (e.g., Command R) is loaded into GPU memory. Size: ~30–70GB depending on quantisation.
2. Each enterprise customer's LoRA adaptor is stored on disk. Size: ~10–100MB per customer.
3. When a request arrives, the serving infrastructure loads the appropriate LoRA adaptor onto the base model in memory. Time: milliseconds.
4. The base model + LoRA adaptor processes the request. The adaptor is applied via low-rank matrix multiplication at each attention layer.
5. If the next request is from a different customer, swap the LoRA adaptor.

Why this works: A single GPU cluster running one base model can serve thousands of fine-tuned customer variants without the memory cost of loading thousands of separate full models.



Design 2 | Private Deployment: Meeting Enterprise Data Requirements

Why enterprise customers want private deployment: Regulated industries (finance, healthcare, legal) cannot send customer data to a third-party API. GDPR and HIPAA require data residency and processing guarantees that a shared API cannot provide. Enterprises also want to ensure their proprietary data does not influence the shared model.

Cohere's two deployment modes:

Mode	What it means	Who uses it
Cohere Platform	Shared API, Cohere's infrastructure	Startups, developers, non-regulated enterprises
Private Deployment	Model + serving infra runs in customer's own cloud VPC	Banks, hospitals, defence, legal firms

The engineering challenge of private deployment: Cohere must package its model and serving infrastructure into a deployable artifact that runs in a customer's AWS/Azure/GCP account. This requires containerised deployment, model weight distribution (secure transfer of large files), and remote monitoring without data egress.

Design 3 | Embedding at Scale: GPU + ANN Cluster Architecture

Use case: Enterprise semantic search. A customer has 100 million documents. They want to embed all documents and serve sub-100ms similarity search.

The scale problem: 100 million documents × 1,024-dimensional embedding × 4 bytes = ∼**400GB** just for the embedding vectors. This does not fit in GPU memory. It does not fit in a single machine's RAM.

Cohere's architecture:

1. **Batch embedding pipeline:** GPU cluster (A100s) runs Cohere Embed over all documents in parallel. At ∼10,000 documents/second per GPU, embedding 100M documents takes ∼3 hours on 4 GPUs.
2. **ANN index construction:** Embeddings written to FAISS or Pinecone. Index construction is a separate compute job.
3. **Online serving:** Query embedding runs on GPU (fast, single vector). Similarity search runs on the ANN cluster (CPU-based, distributed). The two clusters are *separate* because their access patterns and scaling requirements differ.
4. **Incremental updates:** New documents are embedded and added to the index asynchronously. The index does not need to be rebuilt from scratch for small additions.

Design 4 | Multi-Region Deployment: Latency and Data Residency

Concern	Problem	Solution
User latency	UK user hitting US API: +100ms network latency	Regional deployments (US-East, EU-West, Asia-Pacific)
Data residency	GDPR: EU user data must stay in EU	EU-West deployment; data never crosses region boundary
Availability	US-East outage: all users affected	Route UK traffic to EU-West; adds latency, maintains availability

Key Architectural Patterns: Student Design vs. Production Reality

Common student design	What OpenAI / Cohere built	The lesson
Stateful session management (server stores chat history)	Fully stateless — context passed in every request	Statelessness enables horizontal scaling. The cost is larger request sizes.
Single model for all users	Tiered models (3.5 / 4) + model routing	Cost-performance tiering is an infrastructure decision, not just a product decision
One fine-tuned model per enterprise customer	LoRA adaptors on a shared base model	Fine-tuning for N customers requires N adaptors, not N model copies
Rate limiting by requests	Token bucket by token count	LLM requests vary 100× in size; tokens are the right unit of resource consumption
No caching (recompute everything)	Prompt prefix caching (KV cache reuse)	Up to 80% cost reduction for applications with static system prompts